

Spec: Java 27 Learning Curriculum

Status: Delivered, then amended — see *Amendment 1* below **Original scope:** all 24 tasks complete, verified 2026-09-23 **Date:** 2026-09-23 **Phase:** 1 (Specify) of Specify → Plan → Tasks → Implement

Assumptions I'm Making

1. The learner already programs in another language (Python, JavaScript, C#, ...) and is new to **Java specifically**. Modules do not re-teach what a variable or a loop is; they teach how Java does it and where Java differs.
2. "Takeout" in the original request means **Key Takeaways** — a short recap block closing each module.
3. Content and all code are in **English**. This spec is a repository document, also in English.
4. The curriculum covers the **core language and the JDK standard library only**. No Spring, no JDBC, no build frameworks.
5. Lessons and their example code live **together inside each module folder**. Homework solutions live in a **separate top-level solutions/ tree** so the learner is not spoiled by accident.
6. No build tool. Examples run through single-file source launch: `java Example.java`.

→ Correct any of these now, or I proceed with them.

Objective

Build a self-paced Java curriculum that takes a programmer who knows another language and makes them fluent in modern Java, running on JDK 27.

User: A developer with general programming literacy and zero Java experience.

What success looks like: The learner works through numbered modules in order. Each module explains one area in a light, plain-spoken tone, backed by runnable example files whose comments teach the *why* rather than restating the syntax. Each module closes with Key Takeaways and a homework assignment; reference solutions exist but sit apart from the lesson.

A finding that shapes this curriculum

JDK 27 reached General Availability on **15 September 2026**. It is **not an LTS release** (JDK 25 is the current LTS). Its delivered JEPs are almost entirely runtime-level:

JEP	Title	Status
523	Make G1 the Default Garbage Collector in All Environments	Final
527	Post-Quantum Hybrid Key Exchange for TLS 1.3	Final
534	Compact Object Headers by Default	Final
536	JFR In-Process Data Redaction	Final
531	Lazy Constants	Third Preview
532	Primitive Types in Patterns, instanceof, and switch	Fifth Preview
533	Structured Concurrency	Seventh Preview
537	Vector API	Twelfth Incubator
538	PEM Encodings of Cryptographic Objects	Third Preview

Consequence: "Learning Java 27" does not mean learning nine new language features. Nothing in that list changes how a beginner writes code. The honest framing, and the one this curriculum adopts: *JDK 27 is the runtime and toolchain; the language you learn is the stable Java accumulated through JDK 25 LTS*. One closing module covers what 26 and 27 actually changed, why non-LTS releases matter less than release notes suggest, and how to read a JEP.

The on-ramp decision

JEP 512 (Compact Source Files and Instance Main Methods) was **finalized in JDK 25**, so it is stable on 27. Module 01 therefore opens with:

```
void main() {
    IO.println("Hello, World!");
}
```

No `public class`, no `static`, no `String[] args`, no import. Module 05 then deliberately "graduates" the learner to the classic form, because every real codebase, tutorial, and Stack Overflow answer they will ever meet uses it. Teaching the easy form and never explaining the classic one would leave them unable to read real Java.

Tech Stack

Item	Choice
JDK	OpenJDK 27 (GA 2026-09-15)
Language level	Stable features through JDK 25; preview features only in the final module, clearly labeled
Build tool	None
Dependencies	None
Lesson format	Markdown
Example format	Single-file <code>.java</code> , runnable directly

Commands

```
# Run a single example
java modules/01-getting-started/examples/HelloWorld.java

# Run an example that needs preview features (final module only)
java --enable-preview --source 27 modules/22-whats-new/examples/StructuredDemo.java

# Verify every example in the repo compiles and runs
./scripts/verify-examples.sh

# Verify a single module
./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching

# Check the active JDK is 27
java -version
```

Project Structure

```
Java-27-edu/
├── README.md           → Table of contents, how to start, JDK setup
├── SPEC.md            → This document
├── scripts/
│   └── verify-examples.sh → Compiles and runs every example, reports failures
├── modules/
│   ├── 01-getting-started/
│   │   ├── README.md   → Lesson prose + Key Takeaways
│   │   ├── examples/
│   │   │   ├── HelloWorld.java
│   │   │   └── CompileVsRun.java
│   │   └── homework/
│   │       └── README.md → Assignment brief and acceptance criteria
│   ├── 02-variables-and-types/
│   ├── ...
│   └── solutions/
│       ├── 01-getting-started/
│       │   └── Solution.java → Reference solution, kept out of the lesson tree
│       └── ...
```

Module ids are kebab-case, zero-padded, and fixed once assigned. They are never renamed mid-project.

Module README template

```
# Module NN: Title

## What You'll Learn
[3-5 bullets]

## Coming From Another Language
[What this looks like in Python/JS/C# vs. how Java does it]

## The Lesson
[Prose, light in tone, broken by runnable examples]

## Run It
[Exact commands for this module's examples]

## Common Mistakes
[Real traps, with the error message the learner will actually see]

## Key Takeaways
[4-6 bullets – the "takeout"]

## Homework
[Link to homework/README.md]
```

Curriculum Outline

#	Module id	Covers
01	getting-started	JDK 27 setup, JVM/JRE/JDK, compact source files, <code>IO.println</code> , compile vs. run
02	variables-and-types	Primitives vs. references, <code>var</code> , autoboxing traps, String immutability, text blocks
03	control-flow	Operators, <code>if / else</code> , switch expressions, loops, enhanced <code>for</code>
04	methods	Overloading, varargs, pass-by-value-of-references, static vs. instance
05	classes-and-objects	Classes, constructors, encapsulation, the classic <code>main</code> , flexible constructor bodies
06	inheritance-and-polymorphism	<code>extends</code> , <code>super</code> , <code>@Override</code> , <code>abstract</code> , <code>equals</code> / <code>hashCode</code> / <code>toString</code>
07	interfaces	Interfaces, default/static/private methods, functional interfaces
08	records-and-enums	Records, compact constructors, enums with behavior, choosing between them
09	sealed-and-pattern-matching	Sealed types, <code>instanceof</code> patterns, switch patterns, record patterns, exhaustiveness
10	exceptions	Checked vs. unchecked, try-with-resources, custom exceptions, what not to do
11	generics	Generic classes and methods, bounds, wildcards, type erasure, PECS
12	collections	List/Set/Map/Deque, sequenced collections, the equals/hashCode contract, immutable factories
13	lambdas	Lambda syntax, method references, the standard functional interfaces, capture rules
14	streams	Pipelines, intermediate vs. terminal ops, collectors, gatherers, parallel-stream caveats
15	optional	<code>Optional</code> done right, and the anti-patterns that make it worse than null
16	files-and-io	<code>Path</code> , <code>Files</code> , reading and writing text, try-with-resources applied
17	standard-library-tour	Strings, formatting, <code>java.time</code> , <code>Math</code> , random, comparators
18	concurrency-basics	Threads, races, <code>synchronized</code> , atomics, <code>ExecutorService</code>
19	virtual-threads	Virtual threads, scoped values, why blocking is cheap again
20	packages-and-modules	Packages, classpath, module system basics, module import declarations, <code>jar</code> , <code>javadoc</code>
21	testing-your-code	Assertions with <code>-ea</code> , a dependency-free micro test harness, what JUnit adds
22	whats-new	What JDK 26 and 27 actually changed, LTS vs. non-LTS, reading JEPs, preview features

Code Style

One concept per file. Comments teach the *why*; they never narrate the syntax.

```
// Module 02 – examples/AutoboxingTrap.java
//
// Java has two parallel worlds for numbers: primitives (int) and objects
// (Integer). Most of the time Java quietly converts between them, which is
// convenient right up until the moment it bites you. Here is the bite.

void main() {
    Integer a = 127;
    Integer b = 127;
    IO.println(a == b);    // true – and this is the misleading one

    Integer c = 128;
    Integer d = 128;
    IO.println(c == d);    // false! Same value, different objects.

    // Why: Java caches Integer objects for -128..127, so `a` and `b` are
    // literally the same object. Past 127 you get fresh objects, and `==`
    // compares identity, not value.
    //
    // The rule that saves you: use == for primitives, .equals() for objects.
    IO.println(c.equals(d)); // true – what you actually meant
}
```

Conventions: standard Java naming (`PascalCase` types, `camelCase` members, `UPPER_SNAKE` constants), 4-space indent, no line over 100 characters, every example runnable on its own with no arguments.

Testing Strategy

There is no application to unit-test here; the deliverable is teaching material. Correctness means **every example actually runs**.

- `scripts/verify-examples.sh` walks `modules/**/examples/*.java` and `solutions/**/*.*.java`, runs each one, and fails on any non-zero exit or compile error.
- Examples that demonstrate a compile error on purpose carry a `// EXPECT: compile-error` header and are asserted to fail, not skipped.
- Examples needing preview flags carry `// EXPECT: preview` and are run with `--enable-preview -source 27`.
- A module is not "done" until its examples, its homework brief, and its reference solution all pass the script.

- Module 21 teaches testing using `assert` and a hand-rolled harness, keeping the zero-dependency rule intact, and explains what a real JUnit setup would add.
-

Boundaries

Always

- Write all lesson prose, code, comments, and identifiers in English.
- Give every module a Key Takeaways section and a homework assignment.
- Run every example before marking a module complete.
- Show the actual error message a learner will hit when covering a common mistake.
- State plainly when a feature is preview, incubating, or JDK-version-gated.

Ask first

- Adding any third-party dependency, including JUnit.
- Introducing Maven or Gradle.
- Changing approved module ids, their order, or the curriculum outline.
- Expanding scope to frameworks, Android, or web development.

Never

- Ship an example that does not compile without marking it as intentional.
 - Invent JEP numbers, release dates, or feature claims. Verify against openjdk.org.
 - Use jargon before defining it.
 - Present a deprecated or discouraged practice without labeling it as such.
 - Put homework solutions inside the module folder.
-

Success Criteria

All eight checked on 2026-09-23 against the delivered repository. Each line records how it was verified, not merely that it was.

- ✓ 1. `./scripts/verify-examples.sh` exits zero across the whole repository. 130 files, all behaving as declared, in about 70 seconds.
- ✓ 2. All 22 module folders exist, each containing `README.md`, `examples/` with at least two runnable files, and `homework/README.md`. Checked by script across every module directory. 105 example files total, minimum 4 per module.

- ✓ **3. Every module README contains all seven template sections, including Key Takeaways.**
Checked by grepping each heading in all 22 files. One gap was found during the final pass, in `22-whats-new`, and fixed.
- ✓ **4. Every homework has a matching reference solution under `solutions/<module-id>/` that runs clean.** 22 solution directories for 22 modules, 25 solution files, all included in the verification run above.
- ✓ **5. Root `README.md` links every module in order and documents JDK 27 setup.** All 22 links confirmed present by script. Setup documented for macOS, Linux and Windows, with `JAVA_HOME` instructions for each.
- [~] **6. No file contains non-English prose, identifiers, or comments. Superseded by Amendment 1.**
1. True for the English curriculum and for all code, which is what the criterion was protecting. No longer true of the repository as a whole, because `tr/` now holds a Turkish translation of every lesson. Replaced by criteria 9 through 12.
- ✓ **7. No third-party dependency appears anywhere in the repository.** No `pom.xml`, `build.gradle` or `.jar` exists. Every import across all 130 files resolves to `java.*`, `javax.*` or `jdk.*`, plus the JEP 511 `import module java.base` declaration, which is a language feature rather than a dependency.

****Amendment 1 note.**** `scripts/build-pdfs.sh` requires pandoc and a Chromium-based browser. These are external **tools** invoked to render documentation, not libraries the curriculum's code links against. Nothing a learner compiles or runs depends on them, and the PDFs are optional output. The criterion stands as written for the Java code, and this note exists so the distinction is stated rather than assumed.
- ✓ **8. Every factual claim about a JDK version traces to openjdk.org release data.** 32 distinct openjdk.org links across the module and root READMEs. The JDK 26 and 27 JEP tables, the JDK 25 finalisation claims, and JEP 491's removal of virtual thread pinning were each fetched from the source during writing rather than recalled.

Open Questions, as resolved

1. **JDK 27 installation.** Resolved. JDK 27 (build 27+35-2325) was installed before implementation began, and every example has been verified against it.
2. **Module 21 and JUnit.** Resolved in favour of the zero-dependency rule. Testing is taught with bare `assert` plus a 39-line hand-rolled harness, and the module makes the case for adopting JUnit on any real project. The constraint turned out to improve the lesson: seeing a whole working harness explains what a framework does better than using one.
3. **Preview features.** Resolved as proposed. Structured concurrency (JEP 533) and primitive type patterns (JEP 532) appear only in Module 22, labelled with their JEP number and preview round, and

run behind `--enable-preview`. The module argues explicitly against shipping preview features, since a preview class file is rejected by a different release even with the flag.

Corrections Made During Implementation

Recorded because the spec's boundary forbade inventing claims, and holding to that surfaced several that were wrong:

- A composition example asserted that subclassing `ArrayList` would double-count additions. Running it disproved that: modern `ArrayList.addAll` copies in bulk. `HashSet` is the collection that exhibits the bug, and the example now shows both, which makes the fragile-base-class point better than the original would have.
- Three quoted compiler-error line numbers were wrong. All were corrected by producing the errors.
- A bucket-index demonstration used 8 buckets, and `Integer.MIN_VALUE` is divisible by 8, so both the broken and correct forms returned `0` and the bug was invisible. Changed to 7.
- A homework's stated output disagreed with what its reference solution printed. Homework outputs are now diffed against solution output rather than eyeballed.
- The structured concurrency preview history was written from recollection as seven rounds. Reading JEP 533's History section showed two earlier incubator rounds, making it nine releases.

Amendment 1 — PDF output and a Turkish translation

Date: 2026-09-23, after the original 24 tasks were delivered.

Two capabilities were added after the spec was signed off, and neither appeared in any spec or task list at the time it was built. The `spec-driven-development` skill names that exact situation as a red flag: *"Implementing features not mentioned in any spec or task list."* This amendment brings the document back in line with what was actually delivered.

What was added

Capability	What it is
PDF output	<code>scripts/build-pdfs.sh</code> renders every Markdown file to PDF, plus one combined book per language. <code>scripts/pdf.css</code> styles them for print.
Bilingual content	<code>tr/</code> holds a Turkish translation of all 48 Markdown files, plus <code>tr/CEVIRI-NOTLARI.md</code> recording the translation rules.

What the translation deliberately does not translate

Recorded here because it is a constraint, not a preference, and the original spec's Boundaries section had no rule covering a second language:

- **Code blocks stay verbatim English.** They quote the real files under `examples/`, so translating their comments would break the correspondence with what the learner actually runs.
- **Compiler output stays verbatim.** A translated error message is unsearchable and does not match what the terminal prints.
- **Commands, file paths, Java API names and JEP numbers stay as-is.**
- **No Java source lives under `tr/`.** Both languages share the same `modules/*/examples/` and `solutions/` trees.

A violation this amendment caught

Stating the rule above forced a check against it, and the check failed. **Thirty-five Java code blocks across eighteen Turkish files had their comments translated**, in direct violation of the convention the translation itself documents. They were restored to their English originals, and criterion 10 now guards against a recurrence.

This is the value of writing the constraint down rather than holding it in mind: the rule was clear, it was written, and it was still broken forty-five times before anything checked.

Amended success criteria

Criteria 1 through 5, 7 and 8 stand unchanged. Criterion 6 is superseded.

- ✓ **9. Every English Markdown file has a Turkish counterpart at the same path under `tr/`.** All 48 confirmed by script.
- ✓ **10. No `java` code block in any Turkish file contains non-English text.** Zero violations, after 35 were found and restored.
- ✓ **11. Java code blocks align one-to-one between the two languages.** Every one of the 44 paired files has an identical count, so a reader can follow either version against the same code.
- ✓ **12. No Java source file exists under `tr/`.** Zero. Both languages share one set of examples and solutions, so the verification script covers both.
- ✓ **13. `scripts/build-pdfs.sh` produces both languages.** 49 English PDFs in `pdf/`, 50 Turkish in `pdf-tr/`, each with a combined book, in about 100 seconds.
- ✓ **14. The translation does not affect example verification.** `./scripts/verify-examples.sh` still reports all 130 files behaving as declared.

Process note

The correct order would have been to amend this spec **before** building either capability, then re-derive the plan and tasks from it. That did not happen: both were built directly on request and the spec was left stale, with criterion 6 reading ☐ while it was false.

Writing the amendment is what surfaced the code-block violation, which had otherwise shipped. The lesson is the one this curriculum's Module 21 makes about tests: a rule nobody checks is a rule that is already broken.